

Bartosz Gontarek — Project Portfolio

Selected builds · 2025–2026 · AI-first development

Everything below was designed and built by me using an AI-first workflow (Claude Code, Cursor, Gemini) with **Model Context Protocol (MCP)** to connect agents to live data. Code samples are **sanitised** — all API keys/secrets removed and replaced with environment variables.

Positioning: my core priority is **AI & automation engineering** (projects 1–7 below — all active works-in-progress). **Branded e-commerce** (final section) is an **additional income stream** I run in parallel, not my main focus.

1 · Polymarket Trading Bots

Python · Polymarket Gamma & CLOB API · Polygon/Web3 (py-clob-client) · Simmer API · Linux VPS + cron · Telegram

THE PROBLEM

Prediction markets misprice fast-moving events. I wanted to find out, with real code and real data, whether a retail-sized account can extract an edge — and which strategies survive contact with the market.

WHAT I BUILT

- **Copy-trader** — tracks top on-chain wallets, detects *new* positions via the Polymarket data API and mirrors them at a fixed % of balance with exposure caps.
- **AI-divergence scanner** — trades only when an independent AI fair-value estimate diverges from the live price by ≥ 10 pp.
- **BeastBot view-count model** — quant model for YouTube-views markets (project 2).
- **Limitless LP bot** — automated liquidity provision on Base chain with its own backtester + monitor.

ENGINEERING

- **Position diffing** — snapshots holdings per wallet and trades only on the delta, never re-copying stale positions.
- **Risk layer** — copy %, min/max trade size, daily-trade and drawdown caps enforced before any order.
- **Resilience** — every call wrapped in timeouts + try/except; the loop self-heals so a flaky API never kills it.

Honest result: run in paper/\$SIM mode first. The most valuable outcome was *negative* — by auditing **~3,380 on-chain trades** I proved naïve copy-trading is unprofitable at retail scale ($\approx -\$222/\text{week}$ on \$300). That killed a bad strategy **before** risking real money and steered the project to model-driven edges.

Demonstrates: real API + Web3 integration, risk management, and the discipline to falsify my own ideas with data.

2 · BeastBot — quant model for YouTube-views markets

Python 3.10+ · SciPy (sigmoid fit) · YouTube Data API v3 · Polymarket Gamma · Normal-CDF pricing · multi-agent consensus

THE EDGE

Early view data follows a **sigmoid growth curve**. I fit it to project the saturation view count, convert to a bucket probability with a Normal CDF, and trade only when the market diverges from my model by ≥ 8 pp.

```
sources/ → live views (YouTube API) + scraper fallback
model/   → growth_curves (sigmoid) · historical baseline · bootstrap CI
hermes/  → 3 agents (api / curve / historical) → weighted consensus
pricer.py → Normal CDF → bucket probabilities
trader.py → main loop      · monitor.py → exit logic
```

- **Calibrated uncertainty** — sigma widens for longer horizons (grounded in view-growth literature).
- **Consensus scoring** — Hermes blends three signals with explicit weights; skips low-confidence markets.
- **Secrets done right** — all keys via `os.getenv()`, nothing hardcoded.

Demonstrates: applied statistics/ML, clean modular architecture, research → deployable risk-bounded system.

3 · "Mrok Otchłani" — HTML5/Canvas ARPG IN PROGRESS

JavaScript (ES6) · HTML5 Canvas · zero dependencies, zero build step

A top-down Diablo-like that runs from a single HTML file: WASD + mouse aim, **procedurally generated pixel-art**, and a **paper-doll system** where equipped gear changes the character's on-screen look.

- **Procedural art** — sprites/tiles generated in code, so the whole game is self-contained.
- **Game loop** — `requestAnimationFrame` update/render split, entity system, collisions, simple AI.
- Shipped **v0.1 reviewed with 0 blocker bugs**; v2 rebuild in progress.

Demonstrates: strong vanilla-JS fundamentals, real-time rendering and game-systems design without a framework.

4 · STRONEVO — web-studio brand & demo system

HTML/CSS/JS · responsive design · branding

My own web-design studio brand: a catalog site plus **three "offline-proof" demo websites** — self-contained client mock-ups that render without a backend so prospects can click a real-feeling site before committing.

- Defined brand, identity and positioning; built catalog + responsive demo sites packaged as a deployable `dist/`.

Demonstrates: end-to-end front-end delivery, design sensibility, productising a service.

5 · AI Agents & Voice — autonomous assistants

Python · Telegram Bot API · cron/VPS · Google Trends + Amazon · Hume Octave (voice) · MCP

- **"Świstak" (Telegram assistant)** — unified human channel for all bots: pushes fills, daily reports, alerts; takes commands.
- **trends_scout** — daily 06:00 UTC agent scanning foreign product trends → Telegram + Obsidian vault, \$0/month.
- **Voice synthesis** — character voices with Hume Octave 2; explored TTS pipelines for conversational agents.
- **MCP integrations** — wired the official Alpaca MCP and image-gen MCPs into agentic workflows so an agent can call live tools directly.

Demonstrates: exactly the "agentic workflows + MCP" skill set — LLMs that *act* (schedule, fetch, decide, notify), not just chat.

6 · Mia Vex — end-to-end AI persona pipeline IN PROGRESS

Gemini 3 Pro Image / Vertex AI · ComfyUI · RunPod (cloud GPU) · LoRA · Hume voice · prompt engineering

A complete, reproducible pipeline for a consistent AI character across image and voice — from a locked persona to publish-ready assets.

- Configured Vertex AI / Gemini image generation (GCP project, service account, org policy) and validated end-to-end.
- Designed the production pipeline: LoRA character consistency on RunPod GPUs → generation → 4-step publication hygiene.
- Synthesised 110+ research sources into a single A-to-Z operating compendium so the workflow is repeatable.

Demonstrates: cloud GPU orchestration, generative-AI tooling beyond chat, research-to-system rigor.

7 · Video Transcriber — autonomous speech-to-text

Python 3.12 · OpenAI Whisper · CUDA/GPU (RTX) · uv · Windows auto-start watcher

Drop a video in a folder → subtitles appear automatically. Prefers existing captions, falls back to **Whisper** on the GPU, writes results into Obsidian.

- Folder-watcher auto-starts at login; GPU-accelerated inference; reproducible env via uv; model size tuned per language.

Demonstrates: practical ML deployment (real ASR on GPU), automation and reproducibility.

★ Branded E-commerce ADDITIONAL INCOME STREAM

WordPress · WooCommerce · Shopify · NextCart/Takedrop · payments (P24/PayU) · InPost/logistics · branding

Alongside my AI work, I design and ship **branded online stores end-to-end** — the whole funnel: product pages and copy, payment + shipping integrations, legal pages and the brand/visual identity. This is an **additional, parallel income stream** — it funds the AI building, which is my main focus.

BUILDS

- **STRONEVO** — my web-studio brand + offline-proof demo sites to win store clients (*active*).
- **Amovo** — dropshipping store on NextCart/Takedrop (FreshSwap line): product cards, policies, InPost + home delivery (*active*).
- **Lokse** — single-product WooCommerce store (fingerprint padlock), custom brand.
- **GhostShell** — *former brand* (anti-peep phone cases), no longer active.

Demonstrates: end-to-end product delivery, conversion and the full commerce stack (payments, logistics, compliance), and brand sensibility.

How I work: AI-first by default (Claude Code + Cursor + Gemini + MCP + subagents) · ship then prove it (paper-test & audit before risking money) · environment-based secrets, never hardcoded · self-documenting projects + Obsidian knowledge base.

Certified (2026): *Anthropic* — Claude Code in Action · MCP: Advanced Topics · Introduction to MCP · AI Fluency (Framework & Foundations, students, educators, nonprofits, Teaching the Framework). *IBM SkillsBuild* — AI Literacy · AI Fundamentals (Foundations · Language & Vision). **11 certificates total.**